# WorkRadar: AI Workflow

How I used AI to go from the brief to a PRD, a Figma design and a working app | Pravarsha - Airtribe AIPM - Cohort 40

## 1. AI Tools I Used

| Tool | What I used it for |
|---|---|
| **Claude (Anthropic)** | Talking through the problem, personas, user journey and metrics. Writing first drafts of the PRD. Writing the React and TypeScript code and the tests. |
| **Claude inside the app** | Writes the daily summary, answers questions in search, and suggests a next step with a draft message. In the hosted demo it uses the viewer's own Claude account. When self hosted it uses the Anthropic API through a small server. |
| **Claude Artifacts** | Hosting the live app so it can be shared with a link. |
| **Playwright** | Clicking through every screen automatically to check for errors, and exporting the screens into Figma ready files. |
| **Figma** | The final design file: 15 editable screens. |
| **Vitest and ReportLab** | 19 automated tests for the scoring, search and actions, and making the PDF documents. |

## 2. Important Prompts

| Purpose | Prompt (shortened) |
|---|---|
| Changing the user | WorkRadar is for product managers, not engineering managers. How is a PM's day in Slack, Jira, GitHub and Email different? What is the one thing they cannot afford to miss? |
| Finding the insight | In what ways does a PM end up blocking their own team? How could a tool show this without nagging? |
| Sample data | Create 22 realistic, linked work items for a Checkout PM on a Friday morning: a live drop in conversion, a decision engineering is waiting on, a big customer renewal at risk, PRs that need product approval, a launch in 6 days, questions from leaders, an experiment result and a rise in support tickets. |
| Building the app | Build a React and TypeScript app with a Today page, an AI summary, a priority feed with filters and counts, search with @person, source: and is: filters, an item page with a decision picker, and assign, done, snooze and nudge with undo. Keep the ranking rule based and easy to explain. |
| Design export | Write a script that turns the live app screens into SVG files with real shapes and text, so Figma opens them as editable layers that match the app. |
| Summary prompt in the app | You are WorkRadar, helping Ananya Rao, a Senior PM... Helping teammates who are blocked is the most valuable thing a PM can do, so list those decisions first unless there is a live incident... Reply only in this JSON format... Only use item IDs from the list above. |

## 3. What AI Created

- First drafts of the problem statement, personas, journey, pain points, metrics and feature list.
- The data structure for work items, the scoring rules, the backup summary and the search logic.
- All the sample data: 10 people, 22 work items, their conversations, the launch checklist and the timeline.
- Around 3,000 lines of React, TypeScript and CSS, the small API server, the GitHub checks and 19 tests.
- The script that exports the 15 Figma screens, including the design system screen.
- Inside the app: the daily summary, next step suggestions with drafts, and answers in search.

## 4. What I Changed

- **I changed who it is for.** The first version was built for engineering managers. I rebuilt it for product managers and made "teammates blocked on you" the main thing the app shows, with its own colour, its own count on the dashboard and a higher score.
- **I added a decision picker.** The AI suggested a comment box. I turned it into clear options plus a short note. One click unblocks the teammate and ticks off the launch checklist.
- **I kept the ranking in simple rules.** I did not let the AI decide the order. The score is easy to check, and the AI only writes the summary. Any item IDs the AI makes up are removed.
- **I made sure the app works without AI.** A basic summary shows straight away. Claude is used only when the PM clicks, which also keeps the cost low.
- **I kept the design and the app in sync.** Instead of drawing screens separately, I exported the Figma screens from the real app.
- **I fixed problems I found while testing:** colours bleeding into titles, summary numbers out of line, items counted twice, unclear snooze options, missing undo, and no way to reconnect after a sync error.

- **I cut scope on purpose.** No updates back to the source tools and no notifications yet, so I could focus on the main daily flow.

## 5. Key Trade offs

| Area | What I chose | What I gave up for now |
|---|---|---|
| Ranking | Simple scoring rules with reasons | Personalised AI ranking (needs real usage data) |
| When AI runs | Only when clicked, with a backup summary | A fresh AI summary every time the page loads |
| Design | Screens exported from the real app | Freeform design exploration first |
| Data | Realistic sample data | Live connections to the real tools |
| Actions | Saved in WorkRadar, with undo | Updating Jira, Slack and GitHub directly |
| Storage | Saved in the browser for each user | A shared team database |
| AI answers | Strict format checked against the data | Longer, more open answers |

## 6. What I Would Do Next

- Connect the real tools with read only access, then add the option to update them from WorkRadar.
- Add thumbs up or down on the ranking so it learns what each PM cares about.
- Send the morning summary to Slack and draft a weekly update for leaders.
- Build a set of real PM days to test how accurate the AI summary is before every change.